

Lista de Bando de dados II

- 1) **Explique cada propriedade ACID e por que elas são fundamentais em sistemas multiusuários.**

garantem que as transações de banco de dados sejam processadas de forma confiável, evitar que acessos simultâneos corrompam os dados

- 2) **explique a função dos comandos BEGIN TRAN, COMMIT e ROLLBACK, e dê um exemplo real onde elas são essenciais.**

BEGIN TRAN inicia uma transação, o COMMIT salva as alterações permanentemente e o ROLLBACK desfaz tudo caso ocorra um erro. exemplo essencial é uma transferência bancária, onde o dinheiro deve sair de uma conta e entrar em outra obrigatoriamente ao mesmo tempo.

- 3) **O que é a variável @@ERROR e qual sua limitação principal?**

é uma variável de sistema que retorna o número do erro da última instrução Transact-SQL executada pelo usuário.

- 4) **Explique os três pilares do modelo de segurança CIA.**

Confidencialidade (acesso apenas por pessoas autorizadas), Integridade (garantia de que a informação não foi alterada indevidamente) e Disponibilidade (acesso garantido sempre que necessário).

- 5) **Compare os mecanismos TRY/CATCH e THROW na tratativa de erros. Quando cada um deve ser utilizado?**

TRY/CATCH é usado para capturar e tratar erros de execução, permitindo que o código continue ou finalize graciosamente. Já o THROW é utilizado para gerar um erro personalizado

- 6) **Explique a diferença entre login e usuário (user) no SQL Server e por que ambos são necessários para acesso ao banco.**

Login: entidade de segurança no nível da instância do servidor

Usuario: É a identidade vinculada a um banco de dados específico

- 7) **Explique o que são Roles.**

são contêineres que agrupam permissões de acesso, facilitando a administração de segurança ao atribuir privilégios a grupos em vez de indivíduos.

- 8) **Qual a diferença entre roles de servidor e roles de banco de dados? Dê um exemplo de uso.**

Roles de servidor: gerenciam permissões globais (como criar bancos)

Roles de banco de dados: gerenciam objetos internos como ler tabelas

- 9) **Escreva uma SP que receba como parâmetro 3 ID's (@id1, @id2, @id3), e exclua estes IDs no contas a receber. Considere:**

a. Todos os registros devem ser excluídos , ou a operação deve ser revertida

b. Se algum dos registros não existir ou acontecer algum erro, a operação deve ser revertida

c. Caso não exista algum dos registros a SP deve mostrar uma mensagem para o usuário.

```

CREATE PROCEDURE pr_ExcluirContas (@id1 INT, @id2 INT, @id3 INT) AS BEGIN
BEGIN TRANSACTION; BEGIN TRY
    IF NOT EXISTS(SELECT 1 FROM ContasReceber WHERE id IN (@id1, @id2,
@id3))
        BEGIN THROW 50000, 'Um ou mais IDs não encontrados.', 1; END
    DELETE FROM ContasReceber WHERE id IN (@id1, @id2, @id3);
    IF @@ROWCOUNT < 3 BEGIN THROW 50000, 'Nem todos os registros existem.',
1; END
    COMMIT; PRINT 'Exclusão realizada com sucesso.';
END TRY BEGIN CATCH
    ROLLBACK; DECLARE @msg NVARCHAR(200) = ERROR_MESSAGE(); PRINT
@msg;
END CATCH END

```

10) Crie um login no SQL Server com autenticação SQL e depois crie um usuário correspondente dentro do banco uvv

```

CREATE LOGIN login_uvv
WITH PASSWORD = 'Senha@123';

-- Selecionar o banco
USE uvv;
GO

-- Criar usuário associado ao login
CREATE USER usuario_uvv
FOR LOGIN login_uvv;
GO

```

11) Conceda permissão de execução em todas as Stored Procedures do banco para o usuário criado na questão anterior.

```

GRANT EXECUTE TO usuario_uvv;
GO

```

12) Remova o direito de SELECT sobre a tabela *empresa* para o usuário criado.

```

DENY SELECT ON empresa TO usuario_uvv;
GO

```

13) Crie uma Role chamada *Role_financeira* e adicione a ela o usuário criado anteriormente. Em seguida, conceda permissões de SELECT e INSERT nas tabelas pagar e receber e empresa.

```
-- criar role
CREATE ROLE Role_financeira;
GO

-- Adicionar usuário à role
ALTER ROLE Role_financeira
ADD MEMBER usuario_uvv;
GO

-- Permissões nas tabelas
GRANT SELECT, INSERT ON pagar TO Role_financeira;
GRANT SELECT, INSERT ON receber TO Role_financeira;
GRANT SELECT, INSERT ON empresa TO Role_financeira;
GO
```

14) O usuário anterior (adicionado a role) pode consultar todas as colunas da tabela empresa? Porque?

```
DENY SELECT ON empresa TO usuario_uvv;
```

15) É possível um usuário (user) estar relacionado a mais de um login? Explique

No SQL Server:

- Um **login** pertence ao nível do servidor.
- Um **user** pertence ao nível do banco de dados.

Cada usuário do banco pode estar associado a apenas **um login**.

Porém:

- Um login pode estar associado a usuários em vários bancos.
- Um usuário pode participar de várias roles.

16) Crie uma SP para validar usuário e senha que seja imune a ataques de sql inject

```
-- 16) Procedure segura contra SQL Injection
-- Exemplo de validação de usuário e senha
```

```
CREATE PROCEDURE sp_validar_login
(
  @usuario VARCHAR(50),
  @senha VARCHAR(50)
)
AS
BEGIN
  SET NOCOUNT ON;

  SELECT id_usuario,
```

```
nome_usuario  
FROM usuarios  
WHERE nome_usuario = @usuario  
AND senha = @senha;  
END;  
GO
```

E imune a ataques pelo uso de parâmetros (@usuario,@senha)

EXTRAS

Sobre LOGIN e USER no SQL Server, assinale a alternativa correta.

- A) LOGIN existe dentro do banco e USER existe na instância**
- B) LOGIN e USER são exatamente o mesmo objeto**
- C) USER é usado para autenticação na instância**
- D) LOGIN pertence ao servidor e USER pertence ao banco**
- E) Um banco não pode possuir USERS sem LOGIN**

3. `GRANT SELECT ON SCHEMA::financeiro TO joao`

O que ele faz?

- A) Cria o schema financeiro**
- B) Permite SELECT em todas as tabelas do schema financeiro**
- C) Transfere ownership do schema**
- D) Remove permissões do usuário joao**
- E) Cria um login chamado financeiro**

concede a permissão de leitura de forma agrupada; qualquer tabela ou view que pertença ao schema "financeiro" poderá ser consultada pelo usuário "joao" sem a necessidade de permissões individuais

4. Qual comando remove um USER de uma ROLE?

Usa o comando `ALTER ROLE [NomeDaRole] DROP MEMBER [NomeDoUsuario]`. Antigamente, usava-se a procedure `sp_droprolemember`, mas o padrão atual de DDL é o `ALTER ROLE` por ser mais intuitivo e seguir as normas ANSI.

5. Explique detalhadamente a diferença conceitual e funcional entre LOGIN, USER e ROLE no Microsoft SQL Server. Em sua resposta, descreva

Login: Entidade de segurança no nível da instância do servidor

User: É a identidade vinculada a um Login dentro de um banco de dados específico

Role: funciona como um perfil ou papel, que agrupa diversas permissões e as aplica a vários usuários simultaneamente

6. Considere o cenário abaixo no Microsoft SQL Server:

```
CREATE LOGIN gerente_login WITH PASSWORD = '123';
USE uvv;
CREATE USER gerente FOR LOGIN gerente_login;
CREATE TABLE clientes (
    id INT,
    nome VARCHAR(100)
);

GRANT SELECT ON clientes TO gerente;
DENY INSERT ON clientes TO gerente;

--depois
SELECT * FROM clientes;

INSERT INTO clientes VALUES (1, 'João');
```

- quais comandos serão permitidos;
- quais serão negados;
- como o SQL Server resolve permissões conflitantes.
- **Comandos Permitidos:** O comando `SELECT * FROM clientes;` será executado com sucesso, pois poder ter uma concessão explícita de acesso (`GRANT`) para o usuário gerente.
- **Comandos Negados:** O comando `INSERT INTO clientes VALUES (1, 'João');` será bloqueado pelo SQL Server, resultando em um erro de permissão insuficiente.
- **Resolução de Conflitos:** O SQL Server segue o princípio do "Acesso Mínimo", onde um **DENY** (negado) sempre tem precedência sobre um **GRANT** (concedido), independentemente da ordem em que foram aplicados.

7) Um analista precisa consultar todas as tabelas de um banco, mas não deve possuir permissão de alteração (`INSERT`, `UPDATE` ou `DELETE`).

8) Um DBA identificou que um usuário chamado `operador` recebeu permissões excessivas no banco.

O objetivo agora é impedir que esse usuário altere dados da tabela `clientes`, mas mantendo a permissão de leitura

9) A empresa está no período de fechamento de notas. O usuário `ana` já existe no banco `uvv`. Durante três dias, ela precisa consultar e atualizar a tabela `contas_pagar`, mas depois deverá voltar a ter apenas leitura.

Crie os comandos para:

- a) criar uma role chamada `fechamento_notas`;
- b) conceder `SELECT` e `UPDATE` sobre `contas_pagar` para essa role;
- c) inserir `ana` nessa role;
- d) depois remover `ana` da role, sem excluir o usuário

-- Selecionar banco

```
USE uvv;  
GO
```

-- a) Criar a role

```
CREATE ROLE fechamento_notas;  
GO
```

-- b) Conceder `SELECT` e `UPDATE` na tabela `contas_pagar`

```
GRANT SELECT, UPDATE  
ON contas_pagar  
TO fechamento_notas;  
GO
```

-- c) Adicionar `ana` à role

```
ALTER ROLE fechamento_notas  
ADD MEMBER ana;  
GO
```

Depois para remover

```
USE uvv;  
GO
```

-- d) Remover `ana` da role sem excluir o usuário

```
ALTER ROLE fechamento_notas  
DROP MEMBER ana;  
GO
```

10) O usuário `carlos` pertence à role `leitura_geral`, que possui `SELECT` na tabela `clientes`.

Agora ele também deve poder inserir novos clientes, mas não alterar nem excluir registros existentes.

Crie os comandos para:

```
USE uvv;  
GO
```

```
-- Conceder apenas INSERT na tabela clientes  
GRANT INSERT  
ON clientes  
TO carlos;  
GO
```

11) O usuário `marcos` faz parte da role `db_datareader`, mas não deve visualizar a tabela `salarios`.

```
USE uvv;  
GO
```

```
-- Negar SELECT na tabela salarios  
DENY SELECT  
ON salarios  
TO marcos;  
GO
```